



Universidade Federal do Ceará
Campus Quixadá

QXD0011 - Fundamentos de Banco de Dados
Prof. Francisco Victor da Silva Pinheiro

Trabalho Final

Entrega 3 – Modelo Relacional e Implementação no SGBD

Quixadá-CE
2026

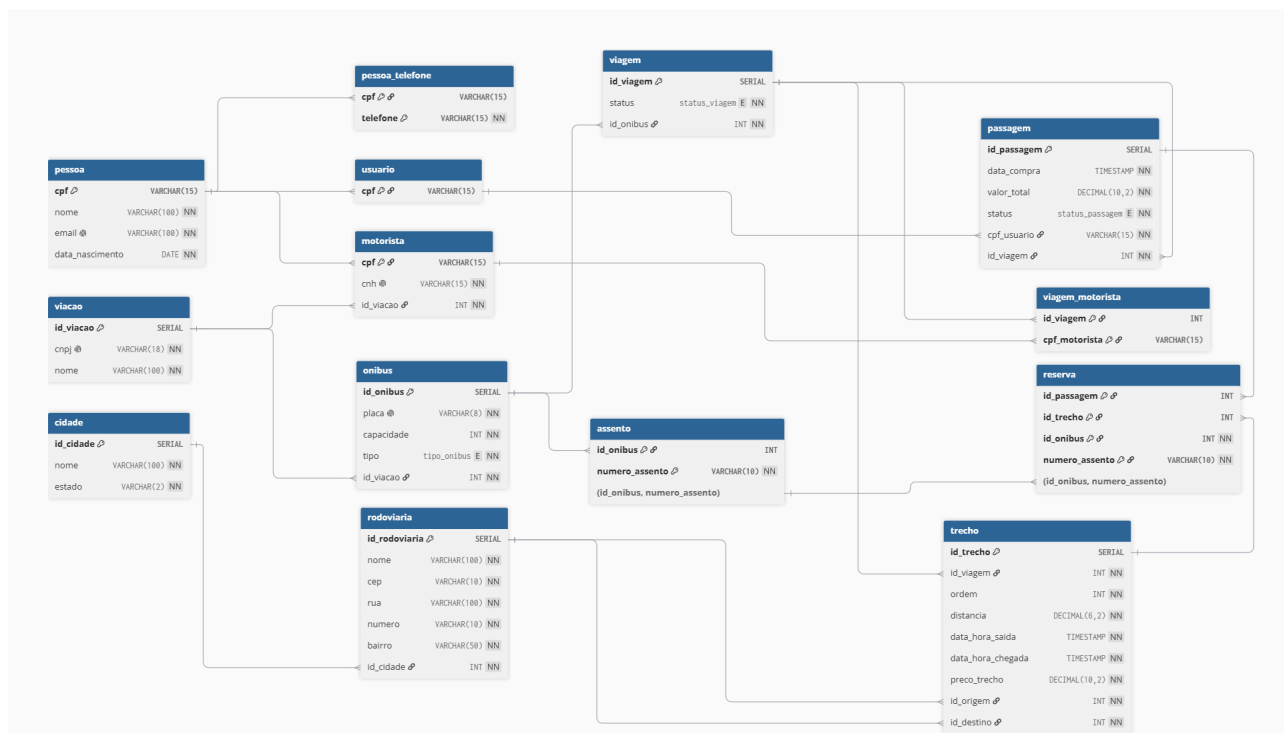
1. Integrantes da Equipe

Matrícula	Nome	email
578795	ANTONIO MARCELO DE MENEZES SILVA	mnzes.marcelo@gmail.com
578984	JOAO VITOR DOURADO FREIRE SANTOS	joaovitor20062006@gmail.com
578168	ARTUR DE SOUSA LIMA	artursousa092@gmail.com
582635	ANTONIO KAUA ALVES DO NASCIMENTO	kaua.alves@alu.ufc.br
582110	GUILHERME WARLEY BRITO FARIAS	warley@alu.ufc.br

2. Resumo da Proposta

Desenvolver e validar a arquitetura do nosso banco de dados relacional para o **Sistema Unificado de Reserva e Venda de Passagens Rodoviárias**. A partir do modelo ER da parte 2, foi realizado o mapeamento completo do banco de dados relacional da aplicação. O banco de dados foi criado no SGBD, garantindo a integridade do modelo, consistência dos dados e inserindo os dados correspondentes. também foram formuladas as consultas SQL que demonstram o funcionamento da aplicação.

3. Mapeamento ER → Relacional



4. Implementação no SGBD (SQL)

- **Criação do esquema (DDL)**

Para traduzir o nosso Modelo Entidade-Relacionamento em um banco de dados físico funcional, nós desenvolvemos o script de DDL (Data Definition Language) focado em três pilares fundamentais: integridade referencial robusta, restrições nativas de consistência e otimização do domínio de dados. utilizando comandos ddl CREATE TYPE e CREATE TABLE.

A primeira etapa do DDL foi criar tipos numerados customizados (**ENUM**). Em vez de permitir que atributos como o estado de uma passagem ou o conforto de um veículo fossem armazenados como texto livre, o próprio banco de dados restringe os valores aceitáveis:

- tipo_onibus: Limita as categorias da frota aos padrões de mercado (Convencional, Executivo, Semi-Leito, Leito, Leito-Cama).
- status_viagem: Controla o ciclo operacional da rota (Agendada, Em Andamento, Concluída, Cancelada, Atrasada).
- status_passagem: Gerencia o fluxo do bilhete (Pendente, Confirmada, Cancelada, Reembolsada, Utilizada).

No modelo conceitual, tínhamos a entidade genérica Pessoa que se especializou em Motorista e Usuário. No DDL, aplicamos a estratégia de **Tabelas de Subclasse com Chaves Compartilhadas**:

- A tabela (pessoa) centraliza os dados comuns (CPF, Nome, E-mail e Data de Nascimento).
- As tabelas (motorista e usuário) possuem como Chave Primária o próprio cpf, mas este campo também é configurado simultaneamente como uma Chave Estrangeira (REFERENCES pessoa (cpf)). Isso garante que nenhum motorista ou usuário exista sem um registro base em pessoa, eliminando redundâncias.

- **Inserção e atualização de dados (DML)**

Tentamos exaustivamente colocar dados reais, com endereços trazidos do google maps, distâncias, cep, quantidade e preço correto de trechos divididos por viagens reais (guanabara por exemplo), porém pela quantidade massiva de dados, sempre gerava o dml incorreto, às vezes por não completar todas as inserções nas tabelas, as vezes por duplicar chaves primárias. Por fim, em um último apelo, com o tempo quase esgotando, geramos um script genérico para criar os dados de forma aleatória, como é descrito logo a seguir.

Os dados foram gerados de forma automatizada e aleatória, com o objetivo de simular um ambiente real de um sistema rodoviário. Foi utilizada geração por scripts para garantir consistência entre as tabelas e suas relações.

Foram criados registros de pessoas com dados fictícios, a partir dos quais derivaram-se usuários e motoristas. Telefones foram associados às pessoas, permitindo múltiplos contatos por registro.

As viagens foram geradas com nomes plausíveis do setor de transporte, servindo como base para a operação dos ônibus e motoristas.

A base geográfica foi composta por cidades brasileiras de diferentes estados, a partir das quais foram criadas rodoviárias associadas.

Os ônibus foram gerados com placas, tipos e capacidades variadas, vinculados às viagens. Para cada ônibus, foram criados assentos numerados sequencialmente.

As viagens foram associadas aos ônibus e compostas por múltiplos trechos sequenciais, representando rotas entre rodoviárias com informações de distância, horários e valores.

As passagens foram geradas para simular compras realizadas por usuários, e as reservas vinculam passagens, trechos e assentos, representando a ocupação dos lugares.

5. Consultas SQL Desenvolvidas

1. Faturamento total por empresa (viação)

Query para saber qual empresa gerou mais receita com a venda de passagens válidas (confirmadas ou já utilizadas).

```
SELECT
  v.id_viacao,
  v.nome AS empresa,
  COUNT(p.id_passagem) AS total_passagens,
  SUM(p.valor_total) AS faturamento_total
FROM viacao v
JOIN onibus o ON o.id_viacao = v.id_viacao
JOIN viagem vg ON vg.id_onibus = o.id_onibus
JOIN passagem p ON p.id_viagem = vg.id_viagem
WHERE p.status IN ('Confirmada', 'Utilizada')
GROUP BY v.id_viacao, v.nome
HAVING SUM(p.valor_total) > 0
ORDER BY faturamento_total DESC;
```

Explicação: Encadeamos viacao → onibus → viagem → passagem (4 tabelas) para conectar a empresa às passagens vendidas. O IN filtra apenas passagens efetivamente válidas, descartando canceladas/reembolsadas/pendentes. GROUP BY agrupa por empresa, HAVING garante faturamento positivo e ORDER BY ranqueia as empresas.

2. Motoristas com mais viagens concluídas

Query para identificar os motoristas mais produtivos para premiação ou redistribuição de escala.

```
SELECT
    m.cpf,
    pe.nome AS motorista,
    v.nome AS empresa,
    COUNT(DISTINCT vg.id_viagem) AS total_viagens
FROM motorista m
JOIN pessoa pe ON pe.cpf = m.cpf
JOIN viacao v ON v.id_viacao = m.id_viacao
JOIN viagem_motorista vm ON vm.cpf_motorista = m.cpf
JOIN viagem vg ON vg.id_viagem = vm.id_viagem
WHERE vg.status = 'Concluida'
GROUP BY m.cpf, pe.nome, v.nome
HAVING COUNT(DISTINCT vg.id_viagem) >= 5
ORDER BY total_viagens DESC;
```

Explicação: Como motorista herda de pessoa, juntamos para obter o nome. A tabela associativa viagem_motorista resolve o relacionamento N:N entre motoristas e viagens (mais de um motorista pode dirigir a mesma viagem). COUNT(DISTINCT ...) evita contagem duplicada caso a junção gere linhas repetidas. HAVING filtra apenas motoristas com volume relevante.

3. Usuários com volume de compras acima da média geral

Query para identificar clientes "fiéis" (compradores frequentes) que compraram mais passagens do que a média de compras por usuário

```
SELECT
    u.cpf,
    pe.nome,
    COUNT(p.id_passagem) AS qtd_passagens,
    SUM(p.valor_total) AS total_gasto
FROM usuario u
JOIN pessoa pe ON pe.cpf = u.cpf
JOIN passagem p ON p.cpf_usuario = u.cpf
WHERE p.status IN ('Confirmada', 'Utilizada')
GROUP BY u.cpf, pe.nome
HAVING COUNT(p.id_passagem) > (
    SELECT AVG(qtd)
    FROM (
        SELECT COUNT(*) AS qtd
```

```

        FROM passagem p2
        WHERE p2.status IN ('Confirmada', 'Utilizada')
        GROUP BY p2.cpf_usuario
    ) AS sub
)
ORDER BY qtd_passagens DESC;

```

Explicação: A subconsulta interna calcula quantas passagens cada usuário comprou; a subconsulta externa tira a média desses totais. O HAVING da consulta principal compara a contagem de cada usuário com essa média global, identificando quem está "acima da média".

4. Trechos mais lucrativos da malha

Query para saber quais trechos (origem→destino) geram mais receita estimada, considerando o preço do trecho e a quantidade de assentos vendidos.

```

SELECT
    t.id_trecho,
    c.nome AS cidade_origem,
    ro.nome AS rodoviaria_origem,
    rd.nome AS rodoviaria_destino,
    COUNT(r.id_passagem) AS passagens_vendidas,
    (COUNT(r.id_passagem) * t.preco_trecho) AS receita_estimada
FROM trecho t
JOIN reserva r ON r.id_trecho = t.id_trecho
JOIN rodoviaria ro ON ro.id_rodoviaria = t.id_origem
JOIN rodoviaria rd ON rd.id_rodoviaria = t.id_destino
JOIN cidade c ON c.id_cidade = ro.id_cidade
GROUP BY t.id_trecho, c.nome, ro.nome, rd.nome, t.preco_trecho
HAVING COUNT(r.id_passagem) >= 10
ORDER BY receita_estimada DESC;

```

Explicação: reserva é a entidade que efetivamente representa "um assento vendido em um trecho", então contamos linhas de reserva por trecho (não passagem, pois uma passagem pode cobrir vários trechos). Usamos rodoviaria duas vezes (origem e destino) com aliases diferentes. cidade enriquece com a localização. HAVING filtra trechos com volume relevante de vendas.

5. Ônibus com maior taxa de ocupação média

Query para saber quais ônibus estão sendo melhor aproveitados (percentual médio de assentos vendidos em relação à capacidade).

```

SELECT

```

```

o.id_onibus,
o.placa,
v.nome AS empresa,
o.capacidade,
AVG(ocupacao.qtd_reservas) AS media_passageiros_por_trecho,
ROUND(AVG(ocupacao.qtd_reservas) * 100.0 / o.capacidade, 2) AS
taxa_ocupacao_percentual
FROM onibus o
JOIN viacao v ON v.id_viacao = o.id_viacao
JOIN (
    SELECT
        t.id_trecho,
        vg.id_onibus,
        COUNT(r.id_passagem) AS qtd_reservas
    FROM trecho t
    JOIN viagem vg ON vg.id_viagem = t.id_viagem
    LEFT JOIN reserva r ON r.id_trecho = t.id_trecho
    GROUP BY t.id_trecho, vg.id_onibus
) AS ocupacao ON ocupacao.id_onibus = o.id_onibus
GROUP BY o.id_onibus, o.placa, v.nome, o.capacidade
HAVING AVG(ocupacao.qtd_reservas) > 0
ORDER BY taxa_ocupacao_percentual DESC;

```

Explicação: A subquery (tabela derivada ocupacao) calcula, para cada trecho percorrido por cada ônibus, quantos assentos foram reservados (usando LEFT JOIN para não perder trechos sem nenhuma venda). A consulta externa tira a média dessas médias por trecho e converte em percentual de ocupação relativo à capacidade do ônibus.

6. Viagens sem nenhuma passagem vendida

Query para identificar viagens "vazias" (programadas, mas sem nenhum passageiro)

```

SELECT
    vg.id_viagem,
    vg.status,
    o.placa,
    v.nome AS empresa
FROM viagem vg
JOIN onibus o ON o.id_onibus = vg.id_onibus
JOIN viacao v ON v.id_viacao = o.id_viacao
WHERE NOT EXISTS (
    SELECT 1
    FROM passagem p

```

```

WHERE p.id_viagem = vg.id_viagem
)
ORDER BY vg.id_viagem;

```

Explicação: NOT EXISTS é a forma mais segura de verificar a ausência de qualquer passagem vinculada à viagem. A subconsulta é correlacionada — para cada linha de viagem, ela checa se existe pelo menos uma passagem associada.

7. Rodoviárias mais movimentadas (embarques + desembarques)

Query para saber quais terminais rodoviários têm maior fluxo total de trechos (somando embarques como origem e desembarques como destino).

```

SELECT
    r.id_rodoviaria,
    r.nome,
    c.nome AS cidade,
    SUM(movimento.qtd) AS total_movimentos
FROM rodoviaria r
JOIN cidade c ON c.id_cidade = r.id_cidade
JOIN (
    SELECT id_origem AS id_rodoviaria, COUNT(*) AS qtd
    FROM trecho
    GROUP BY id_origem
    UNION ALL
    SELECT id_destino AS id_rodoviaria, COUNT(*) AS qtd
    FROM trecho
    GROUP BY id_destino
) AS movimento ON movimento.id_rodoviaria = r.id_rodoviaria
GROUP BY r.id_rodoviaria, r.nome, c.nome
HAVING SUM(movimento.qtd) >= 5
ORDER BY total_movimentos DESC;

```

Explicação: Como a tabela trecho tem dois papéis para rodoviaria (id_origem e id_destino), usamos UNION ALL para juntar as duas contagens em uma única lista de movimentos antes de agrupar por rodoviária. UNION ALL é usado propositalmente para não eliminar linhas legítimas (uma rodoviária pode ter a mesma contagem de origem e destino).

8. Cidades com maior fluxo de passageiros embarcados

Query para saber quais cidades de origem concentram mais passageiros pagantes, para priorizar investimento em frota/horários.

```

SELECT

```



```

    c.id_cidade,
    c.nome,
    c.estado,
    COUNT(r.id_passagem) AS total_embarques
FROM cidade c
JOIN rodoviaria ro ON ro.id_cidade = c.id_cidade
JOIN trecho t ON t.id_origem = ro.id_rodoviaria
JOIN reserva r ON r.id_trecho = t.id_trecho
JOIN passagem p ON p.id_passagem = r.id_passagem
WHERE p.status IN ('Confirmada', 'Utilizada')
GROUP BY c.id_cidade, c.nome, c.estado
HAVING COUNT(r.id_passagem) > 20
ORDER BY total_embarques DESC;

```

Explicação: Percorremos cidade → rodoviaria → trecho (como origem) → reserva → passagem (5 tabelas) para contar quantos assentos efetivamente vendidos partiram de cada cidade. O filtro IN garante que só contamos passagens válidas, e o HAVING recorta apenas cidades com fluxo relevante.

9. Trechos com preço acima da média geral da malha

Query para saber quais trechos estão precificados acima da média de toda a rede

```

SELECT
    t.id_trecho,
    t.preco_trecho,
    ro.nome AS origem,
    rd.nome AS destino,
    vg.id_viagem,
    v.nome AS empresa
FROM trecho t
JOIN rodoviaria ro ON ro.id_rodoviaria = t.id_origem
JOIN rodoviaria rd ON rd.id_rodoviaria = t.id_destino
JOIN viagem vg ON vg.id_viagem = t.id_viagem
JOIN onibus o ON o.id_onibus = vg.id_onibus
JOIN viacao v ON v.id_viacao = o.id_viacao
WHERE t.preco_trecho > (SELECT AVG(preco_trecho) FROM trecho)
ORDER BY t.preco_trecho DESC;

```

Explicação: A subconsulta escalar não correlacionada calcula a média global de preco_trecho uma única vez; a cláusula WHERE da consulta principal compara cada trecho contra esse valor fixo. As 5 junções adicionam contexto de negócio (de/para, viagem e empresa responsável).

10. Usuários que viajaram para mais destinos distintos

Query para identificar usuários com maior diversidade de destinos

```
SELECT
    u.cpf,
    pe.nome,
    COUNT(DISTINCT t.id_destino) AS destinos_distintos
FROM usuario u
JOIN pessoa pe ON pe.cpf = u.cpf
JOIN passagem p ON p.cpf_usuario = u.cpf
JOIN reserva r ON r.id_passagem = p.id_passagem
JOIN trecho t ON t.id_trecho = r.id_trecho
WHERE p.status IN ('Confirmada', 'Utilizada')
GROUP BY u.cpf, pe.nome
HAVING COUNT(DISTINCT t.id_destino) >= 3
ORDER BY destinos_distintos DESC;
```

Explicação: Navegamos usuario → passagem → reserva → trecho para chegar até o destino (id_destino) de cada trecho efetivamente reservado pelo usuário. COUNT(DISTINCT ...) evita contar o mesmo destino várias vezes caso o usuário tenha viajado repetidamente para o mesmo lugar. HAVING filtra apenas quem visitou 3+ destinos diferentes.

11. Motoristas que trabalharam em viagens de longa distância

Query para saber quais motoristas estão sendo alocados em viagens de longo curso (soma de distância dos trechos acima de um limite)

```
SELECT
    m.cpf,
    pe.nome,
    v.nome AS empresa,
    vg.id_viagem,
    soma.distancia_total
FROM motorista m
JOIN pessoa pe ON pe.cpf = m.cpf
JOIN viacao v ON v.id_viacao = m.id_viacao
JOIN viagem_motorista vm ON vm.cpf_motorista = m.cpf
JOIN viagem vg ON vg.id_viagem = vm.id_viagem
JOIN (
    SELECT id_viagem, SUM(distancia) AS distancia_total
    FROM trecho
    GROUP BY id_viagem
```

```
) AS soma ON soma.id_viagem = vg.id_viagem
WHERE soma.distancia_total > 500
ORDER BY soma.distancia_total DESC;
```

Explicação: Como uma viagem pode ter múltiplos trechos (paradas), a distância "total" da viagem não está armazenada diretamente — é calculada via subconsulta (tabela derivada soma) que agrupa trecho por id_viagem. Essa tabela derivada é unida à cadeia motorista → viagem_motorista → viagem, permitindo filtrar viagens longas (>500 km) e os motoristas envolvidos.

12. Comparação entre empresas: ticket médio acima de todas

Query para saber se existe alguma empresa cujo ticket médio por passagem é superior a **todas** as outras (não apenas à média geral)

```
SELECT
    v.id_viacao,
    v.nome,
    AVG(p.valor_total) AS ticket_medio
FROM viacao v
JOIN onibus o ON o.id_viacao = v.id_viacao
JOIN viagem vg ON vg.id_onibus = o.id_onibus
JOIN passagem p ON p.id_viagem = vg.id_viagem
WHERE p.status IN ('Confirmada', 'Utilizada')
GROUP BY v.id_viacao, v.nome
HAVING AVG(p.valor_total) > ALL (
    SELECT AVG(p2.valor_total)
    FROM viacao v2
    JOIN onibus o2 ON o2.id_viacao = v2.id_viacao
    JOIN viagem vg2 ON vg2.id_onibus = o2.id_onibus
    JOIN passagem p2 ON p2.id_viagem = vg2.id_viagem
    WHERE v2.id_viacao <> v.id_viacao
    AND p2.status IN ('Confirmada', 'Utilizada')
    GROUP BY v2.id_viacao
)
ORDER BY ticket_medio DESC;
```

Explicação: O operador > ALL (subconsulta) exige que o ticket médio da empresa analisada seja maior do que **cada um** dos valores retornados pela subconsulta (que calcula o ticket médio de todas as outras empresas, excluindo a própria via <> correlacionado).

13. Tempo médio de viagem por empresa e tipo de ônibus

Query para comparar a duração média dos trechos por tipo de ônibus (Convencional, Executivo, Leito etc.) e por empresa

```
SELECT
    v.nome AS empresa,
    o.tipo,
    COUNT(DISTINCT t.id_trecho) AS qtd_trechos,
    AVG(t.data_hora_chegada - t.data_hora_saida) AS duracao_media
FROM trecho t
JOIN viagem vg ON vg.id_viagem = t.id_viagem
JOIN onibus o ON o.id_onibus = vg.id_onibus
JOIN viacao v ON v.id_viacao = o.id_viacao
GROUP BY v.nome, o.tipo
HAVING COUNT(DISTINCT t.id_trecho) >= 3
ORDER BY duracao_media DESC;
```

Explicação: Subtraindo data_hora_chegada - data_hora_saida obtemos um INTERVAL por trecho; AVG() sobre intervalos é suportado nativamente e retorna a duração média no formato HH:MM:SS. Agrupamos por empresa e tipo de ônibus simultaneamente para permitir comparações cruzadas (ex.: "Leito da Empresa X é mais rápido que Executivo da Empresa Y?"). HAVING exige amostra mínima de 3 trechos para a média ser estatisticamente relevante.

14. Quantidade total de passageiros transportados por motorista

Query para fazer um ranking de "volume de passageiros conduzidos" por motorista (não apenas número de viagens, mas total de pessoas efetivamente embarcadas)

```
SELECT
    m.cpf,
    pe.nome,
    v.nome AS empresa,
    (
        SELECT COUNT(*)
        FROM reserva r
        JOIN trecho t ON t.id_trecho = r.id_trecho
        WHERE t.id_viagem IN (
            SELECT vm2.id_viagem
            FROM viagem_motorista vm2
            WHERE vm2.cpf_motorista = m.cpf
        )
    ) AS total_passageiros_levados
FROM motorista m
JOIN pessoa pe ON pe.cpf = m.cpf
JOIN viacao v ON v.id_viacao = m.id_viacao
```

```

WHERE EXISTS (
    SELECT 1 FROM viagem_motorista vmx WHERE vmx.cpf_motorista = m.cpf
)
ORDER BY total_passageiros_levados DESC;

```

Explicação: Esta é a consulta com subconsultas em **três níveis**: (1) a subconsulta mais interna lista os id_viagem que o motorista dirigiu (via viagem_motorista); (2) o IN filtra os trechos (trecho) pertencentes a essas viagens; (3) a subconsulta do SELECT conta quantas linhas de reserva (= passageiros embarcados) existem nesses trechos. O EXISTS na cláusula WHERE principal garante que só listamos motoristas que de fato já dirigiram alguma viagem, evitando linhas com contagem zero sem sentido.

15. Melhores rotas (origem → destino) por receita e demanda

Query para saber quais **pares de cidades** (rotas) geram mais receita e movimentam mais passageiros no total, agregando todas as viagens já realizadas nesse trajeto

```

SELECT

    co.nome AS cidade_origem,
    co.estado AS uf_origem,
    cd.nome AS cidade_destino,
    cd.estado AS uf_destino,
    COUNT(DISTINCT t.id_viagem) AS qtd_viagens_realizadas,
    COUNT(r.id_passagem) AS total_passageiros_transportados,
    SUM(t.preco_trecho) AS receita_estimada,
    ROUND(AVG(t.preco_trecho), 2) AS preco_medio_praticado
FROM trecho t
JOIN rodoviaria ro ON ro.id_rodoviaria = t.id_origem
JOIN rodoviaria rd ON rd.id_rodoviaria = t.id_destino
JOIN cidade co ON co.id_cidade = ro.id_cidade
JOIN cidade cd ON cd.id_cidade = rd.id_cidade
JOIN reserva r ON r.id_trecho = t.id_trecho
JOIN passagem p ON p.id_passagem = r.id_passagem
WHERE p.status IN ('Confirmada', 'Utilizada')
GROUP BY co.nome, co.estado, cd.nome, cd.estado
HAVING COUNT(r.id_passagem) >= 10
ORDER BY receita_estimada DESC;

```

Explicação: Essa é a consulta com junções múltiplas (*self-joins*) e agregações para análise de rotas. Os *self-joins* mapeiam simultaneamente a origem e o destino do trecho. O GROUP BY consolida os dados por cidade, unificando diferentes viagens que fazem o mesmo percurso. O SELECT calcula as métricas, contando

as reservas para medir passageiros reais e somando os preços para estimar a receita. O HAVING aliado ao WHERE garante que o ranking considere apenas vendas válidas em rotas consistentes (mínimo de 10 passageiros), evitando que rotas pouco testadas distorçam os resultados.

16. Quantidade de ônibus por empresa

Query para saber rapidamente quantos veículos cada viação possui.

```
SELECT
    v.nome AS empresa,
    COUNT(o.id_onibus) AS qtd_onibus
FROM viacao v
LEFT JOIN onibus o ON o.id_viacao = v.id_viacao
GROUP BY v.nome
ORDER BY qtd_onibus DESC;
```

Explicação: Usa LEFT JOIN para garantir que empresas sem nenhum ônibus cadastrado ainda apareçam no resultado (com contagem zero), e não sejam excluídas como aconteceria com INNER JOIN.

17. Total de cidades cadastradas por estado

Query para saber quantas cidades já estão cadastradas em cada estado

```
SELECT
    estado,
    COUNT(*) AS qtd_cidades
FROM cidade
GROUP BY estado
HAVING COUNT(*) >= 2
ORDER BY qtd_cidades DESC;
```

Explicação: Agregação simples com GROUP BY em uma única tabela. O HAVING filtra apenas estados com pelo menos 2 cidades cadastradas.

18. Ônibus do tipo Leito ou Leito-Cama

Query para trazer todos os ônibus de categoria premium disponíveis na frota.

```
SELECT
    o.placa,
```

```

    o.tipo,
    o.capacidade,
    v.nome AS empresa
FROM onibus o
JOIN viacao v ON v.id_viacao = o.id_viacao
WHERE o.tipo IN ('Leito', 'Leito-Cama')
ORDER BY o.capacidade DESC;

```

Explicação: Filtro com IN sobre o enum tipo_onibus, combinado a um JOIN simples para mostrar a empresa responsável por cada ônibus.

19. Viagens atualmente em andamento

Query para saber quantas viagens estão acontecendo agora e em quais ônibus.

```

SELECT
    vg.id_viagem,
    vg.status,
    o.placa,
    o.tipo,
    v.nome AS empresa
FROM viagem vg
JOIN onibus o ON o.id_onibus = vg.id_onibus
JOIN viacao v ON v.id_viacao = o.id_viacao
WHERE vg.status = 'Em Andamento';

```

Explicação: Três tabelas unidas em cadeia simples (viagem → onibus → viacao) com um filtro direto pelo enum status_viagem.

20. Usuários que nunca compraram nenhuma passagem

Query para identificar usuários cadastrados que nunca efetivaram uma compra

```

SELECT
    u.cpf,
    pe.nome,
    pe.email
FROM usuario u
JOIN pessoa pe ON pe.cpf = u.cpf
WHERE NOT EXISTS (
    SELECT 1
    FROM passagem p
    WHERE p.cpf_usuario = u.cpf
);

```

Explicação: NOT EXISTS verifica, para cada usuário, se não existe nenhuma linha correspondente em passagem. É a forma mais simples e segura de checar "ausência total de registro relacionado".

21. Preço médio dos trechos por tipo de ônibus

Query para comparar o preço médio cobrado conforme a categoria do ônibus (Convencional, Executivo, Leito etc.).

```
SELECT
    o.tipo,
    COUNT(t.id_trecho) AS qtd_trechos,
    ROUND(AVG(t.preco_trecho), 2) AS preco_medio
FROM trecho t
JOIN viagem vg ON vg.id_viagem = t.id_viagem
JOIN onibus o ON o.id_onibus = vg.id_onibus
GROUP BY o.tipo
ORDER BY preco_medio DESC;
```

Explicação: Encadeia três tabelas (trecho → viagem → onibus) e agrupa pelo enum tipo_onibus, usando AVG para comparar preços médios entre as categorias.

22. Rodoviárias de um estado específico

Query para lista de todas as rodoviárias localizadas em um determinado estado (ex: Ceará)

```
SELECT
    r.nome AS rodoviaria,
    r.bairro,
    c.nome AS cidade,
    c.estado
FROM rodoviaria r
JOIN cidade c ON c.id_cidade = r.id_cidade
WHERE c.estado = 'CE'
ORDER BY c.nome, r.nome;
```

Explicação: JOIN simples entre rodoviaria e cidade com filtro direto pela sigla do estado.

6. Anexos

- [Comandos DDL, DML, consultas, modelo-relacional.pdf/png.](#)

7. Melhorias com relação à proposta da Entrega 2 (se houver)

- **Inclusão do atributo Distância na entidade Trecho:** Adicionamos esse atributo porque percebemos, ao criar as queries, que ele é essencial para as operações e regras de negócio do projeto.
- **Ajuste no relacionamento Trecho x Viagem (de N:M para 1:N):** A ideia inicial era de usar um trecho genérico, porém com a tabela associativa geraria redundância e complexidade desnecessária com a entidade Reserva. A mudança para 1:N otimiza o esquema do banco e garante melhor integridade referencial.
- **Correção no relacionamento Usuário x Passagem (de N:M para 1:N):** Corrigimos um erro de documentação do esboço inicial.

SCRIPT DDL -----

----- CRIACAO DO BANCO

-- Enums

```
CREATE TYPE tipo_onibus AS ENUM ('Convencional', 'Executivo', 'Semi-Leito',
'Leito', 'Leito-Cama');
```

```
CREATE TYPE status_viagem AS ENUM ('Agendada', 'Em Andamento',
'Concluida', 'Cancelada', 'Atrasada');
```

```
CREATE TYPE status_passagem AS ENUM ('Pendente', 'Confirmada',
'Cancelada', 'Reembolsada', 'Utilizada');
```

-- Pessoa para heranca

```
CREATE TABLE pessoa (
    cpf VARCHAR(15) PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    data_nascimento DATE NOT NULL
);
```

-- Telefone (mutivalorado)

```
CREATE TABLE pessoa_telefone (
    cpf VARCHAR(15) REFERENCES pessoa(cpf),
    telefone VARCHAR(15) NOT NULL,
```

```

PRIMARY KEY (cpf, telefone)

);

CREATE TABLE viacao (
    id_viacao SERIAL PRIMARY KEY,
    cnpj VARCHAR(18) UNIQUE NOT NULL,
    nome VARCHAR(100) NOT NULL
);

-- Motorista herda de pessoa
CREATE TABLE motorista (
    cpf VARCHAR(15) PRIMARY KEY REFERENCES pessoa(cpf),
    cnh VARCHAR(15) UNIQUE NOT NULL,
    id_viacao INT NOT NULL REFERENCES viacao(id_viacao)
);

-- Usuario herda de pessoa
CREATE TABLE usuario (
    cpf VARCHAR(15) PRIMARY KEY REFERENCES pessoa(cpf)
);

CREATE TABLE onibus (
    id_onibus SERIAL PRIMARY KEY,
    placa VARCHAR(8) UNIQUE NOT NULL,
    capacidade INT NOT NULL,
    tipo tipo_onibus NOT NULL,
    id_viacao INT NOT NULL REFERENCES viacao(id_viacao)
);

```

-- Entidade fraca de ônibus

```
CREATE TABLE assento (  
    id_onibus INT REFERENCES onibus(id_onibus),  
    numero_assento VARCHAR(10) NOT NULL,  
    PRIMARY KEY (id_onibus, numero_assento)  
);
```

```
CREATE TABLE cidade (  
    id_cidade SERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    estado VARCHAR(2) NOT NULL  
);
```

```
CREATE TABLE rodoviaria (  
    id_rodoviaria SERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    cep VARCHAR(10) NOT NULL,  
    rua VARCHAR(100) NOT NULL,  
    numero VARCHAR(10) NOT NULL,  
    bairro VARCHAR(50) NOT NULL,  
    id_cidade INT NOT NULL REFERENCES cidade(id_cidade)  
);
```

```
CREATE TABLE viagem (  
    id_viagem SERIAL PRIMARY KEY,  
    status status_viagem NOT NULL,  
    id_onibus INT NOT NULL REFERENCES onibus(id_onibus)
```

);

-- Tabela associativa para permitir varios motoristas em uma mesma viagem

```
CREATE TABLE viagem_motorista (  
    id_viagem INT REFERENCES viagem(id_viagem),  
    cpf_motorista VARCHAR(15) REFERENCES motorista(cpf),  
    PRIMARY KEY (id_viagem, cpf_motorista)  
);
```

-- Setor especifico da viagem

```
CREATE TABLE trecho (  
    id_trecho SERIAL PRIMARY KEY,  
    id_viagem INT NOT NULL REFERENCES viagem(id_viagem),  
    ordem INT NOT NULL,  
    distancia DECIMAL(6,2) NOT NULL,  
    data_hora_saida TIMESTAMP NOT NULL,  
    data_hora_chegada TIMESTAMP NOT NULL,  
    preco_trecho DECIMAL(10,2) NOT NULL,  
    id_origem INT NOT NULL REFERENCES rodoviaria(id_rodoviaria),  
    id_destino INT NOT NULL REFERENCES rodoviaria(id_rodoviaria)  
);
```

```
CREATE TABLE passagem (  
    id_passagem SERIAL PRIMARY KEY,  
    data_compra TIMESTAMP NOT NULL,  
    valor_total DECIMAL(10,2) NOT NULL,  
    status status_passagem NOT NULL,  
    cpf_usuario VARCHAR(15) NOT NULL REFERENCES usuario(cpf),
```

```

        id_viagem INT NOT NULL REFERENCES viagem(id_viagem)
    );

-- Relacionamento ternário

CREATE TABLE reserva (
    id_passagem INT REFERENCES passagem(id_passagem),
    id_trecho INT REFERENCES trecho(id_trecho),
    id_onibus INT NOT NULL,
    numero_assento VARCHAR(10) NOT NULL,
    FOREIGN KEY (id_onibus, numero_assento) REFERENCES assento(id_onibus,
numero_assento),
    PRIMARY KEY (id_passagem, id_trecho, id_onibus, numero_assento),
    CONSTRAINT assento_unico_por_trecho UNIQUE (id_trecho, id_onibus,
numero_assento)
);

```

SCRIPT DML -----

DO \$\$

DECLARE

-- Arrays para geração de dados realistas

```

    nomes TEXT[] := ARRAY['João', 'Maria', 'José', 'Ana', 'Carlos', 'Antônio',
'Francisco', 'Luiz', 'Paulo', 'Marcos', 'Lucas', 'Pedro', 'Rafael', 'Diego', 'Bruno',
'Rodrigo', 'Fernanda', 'Juliana', 'Camila', 'Aline', 'Amanda', 'Letícia', 'Larissa',
'Bruna'];

```

```

    sobrenomes TEXT[] := ARRAY['Silva', 'Santos', 'Oliveira', 'Souza', 'Rodrigues',
'Ferreira', 'Alves', 'Pereira', 'Lima', 'Gomes', 'Costa', 'Ribeiro', 'Martins', 'Carvalho',
'Almeida', 'Lopes', 'Soares', 'Fernandes', 'Vieira', 'Barbosa', 'Rocha'];

```

-- Cidades do Ceará (40% do peso aproximado)

```
    cidades_ce TEXT[] := ARRAY['Fortaleza', 'Juazeiro do Norte', 'Sobral', 'Crato',  
'Caucaia', 'Maracanaú', 'Iguatu', 'Quixadá', 'Itapipoca', 'Crateús', 'Aracati',  
'Camocim', 'Tianguá', 'Barbalha', 'Russas'];
```

```
-- Cidades do Resto do Brasil (60% do peso)
```

```
    cidades_br TEXT[] := ARRAY['São Paulo/SP', 'Rio de Janeiro/RJ', 'Belo  
Horizonte/MG', 'Salvador/BA', 'Brasília/DF', 'Curitiba/PR', 'Recife/PE', 'Porto  
Alegre/RS', 'Belém/PA', 'Goiânia/GO', 'Manaus/AM', 'Natal/RN', 'João Pessoa/PB',  
'Teresina/PI', 'São Luís/MA', 'Maceió/AL', 'Aracaju/SE', 'Florianópolis/SC',  
'Vitória/ES', 'Cuiabá/MT'];
```

```
    viacoes_nomes TEXT[] := ARRAY['Viação Guanabara', 'Auto Viação Progresso',  
'Expresso Guanabara', 'Viação Cometa', 'Auto Viação 1001', 'Gontijo', 'Águia  
Branca', 'Itapemirim', 'Catarinense', 'Reunidas', 'Real Expresso', 'Rápido Federal'];
```

```
-- Variáveis de controle
```

```
v_cpf VARCHAR(15);
```

```
v_email VARCHAR(100);
```

```
v_cnpj VARCHAR(18);
```

```
v_placa VARCHAR(8);
```

```
v_id_cidade INT;
```

```
v_id_viacao INT;
```

```
v_id_onibus INT;
```

```
v_id_viagem INT;
```

```
v_id_passagem INT;
```

```
v_id_trecho INT;
```

```
-- Variáveis para a lógica das viagens longas
```

```
v_num_trechos INT;
```

```
v_origem_rod INT;
```

```
v_destino_rod INT;
```

```
v_data_atual TIMESTAMP;
```

```
v_distancia DECIMAL(6,2);
```

```

v_preco DECIMAL(10,2);

-- Loops e contadores
i INT;
j INT;
k INT;
t INT;

-- Cursors para buscar chaves estrangeiras
c_motoristas VARCHAR(15)[];
c_usuarios VARCHAR(15)[];
c_rodovias INT[];
c_onibus_lista INT[];
c_trechos_viagem INT[];
BEGIN

-----

-- 1. POPULAR TABELA: pessoa (Gerar ~300 para suprir motoristas e usuários)
-----

FOR i IN 1..300 LOOP

    v_cpf := LPAD(CAST(CAST(random() * 89999999999 AS BIGINT) +
10000000000 AS TEXT), 11, '0');

    v_email := lower(nomes[1 + floor(random() * array_length(nomes, 1))] || i ||
'@email.com');

    INSERT INTO pessoa (cpf, nome, email, data_nascimento)

    VALUES (

        v_cpf,

        nomes[1 + floor(random() * array_length(nomes, 1))] || ' ' || sobrenomes[1 +
floor(random() * array_length(sobrenomes, 1))],

```

```

v_email,

-- CORREÇÃO: DATE em vez de DATA
DATE '1960-01-01' + CAST(random() * 15000 AS INT)
) ON CONFLICT DO NOTHING;
END LOOP;

-----

-- 2. POPULAR TABELA: pessoa_telefone (Multivalorado)
-----

FOR v_cpf IN SELECT cpf FROM pessoa LOOP
    INSERT INTO pessoa_telefone (cpf, telefone)
        VALUES (v_cpf, '(85) 9' || CAST(CAST(random() * 89999999 AS INT) +
10000000 AS TEXT));

    -- Alguns têm dois telefones
    IF random() > 0.6 THEN
        INSERT INTO pessoa_telefone (cpf, telefone)
            VALUES (v_cpf, '(85) 9' || CAST(CAST(random() * 89999999 AS INT) +
10000000 AS TEXT))
            ON CONFLICT DO NOTHING;
    END IF;
END LOOP;

-----

-- 3. POPULAR TABELA: viacao (120 Empresas)
-----

FOR i IN 1..120 LOOP
    v_cnpj := LPAD(i::TEXT, 14, '0');
    INSERT INTO viacao (cnpj, nome)
        VALUES (

```



```

        v_cnpj,
        viacoes_nomes[1 + ((i-1) % array_length(viacoes_nomes, 1))] || ' - Filial ' || i
    );
END LOOP;

-----

-- 4. POPULAR TABELA: motorista (130 motoristas)
-----

i := 1;
FOR v_cpf IN SELECT cpf FROM pessoa LIMIT 130 LOOP
    SELECT id_viacao INTO v_id_viacao FROM viacao OFFSET (i % 120) LIMIT
1;

    INSERT INTO motorista (cpf, cnh, id_viacao)

        VALUES (v_cpf, LPAD(CAST(CAST(random() * 899999999 AS BIGINT) +
100000000 AS TEXT), 11, '0'), v_id_viacao);

    i := i + 1;
END LOOP;

-----

-- 5. POPULAR TABELA: usuario (Restante das pessoas viram usuários, > 150)
-----

INSERT INTO usuario (cpf)
SELECT cpf FROM pessoa WHERE cpf NOT IN (SELECT cpf FROM motorista);

-----

-- 6. POPULAR TABELA: cidade (Cumprindo a regra dos 40% Ceará)
-----

-- 50 Cidades do Ceará (40% de 125)

FOR i IN 1..50 LOOP

```

```

INSERT INTO cidade (nome, estado)

VALUES (cidades_ce[1 + ((i-1) % array_length(cidades_ce, 1))] || ' ' || i, 'CE');

END LOOP;


-- 75 Cidades do Resto do Brasil (60%)
FOR i IN 1..75 LOOP
    INSERT INTO cidade (nome, estado)
    VALUES (
        split_part(cidades_br[1 + ((i-1) % array_length(cidades_br, 1))], '/', 1) || ' ' || i,
        split_part(cidades_br[1 + ((i-1) % array_length(cidades_br, 1))], '/', 2)
    );
END LOOP;


-----

-- 7. POPULAR TABELA: rodoviaria (130 Rodoviárias distribuídas nas cidades)
-----

i := 1;

FOR v_id_cidade IN SELECT id_cidade FROM cidade LOOP
    INSERT INTO rodoviaria (nome, cep, rua, numero, bairro, id_cidade)
    VALUES (
        'Terminal Rodoviário de ' || (SELECT nome FROM cidade WHERE
id_cidade = v_id_cidade),
        LPAD(CAST(CAST(random() * 89999 AS INT) + 10000 AS TEXT), 5, '0') ||
'-000',
        'Av. Central',
        CAST(CAST(random() * 2000 AS INT) + 1 AS TEXT),
        'Centro',
        v_id_cidade
    );

```

```

i := i + 1;

EXIT WHEN i > 130;

END LOOP;

```

```

WHILE (SELECT count(*) FROM rodoviaria) < 130 LOOP

    SELECT id_cidade INTO v_id_cidade FROM cidade ORDER BY random()
LIMIT 1;

    INSERT INTO rodoviaria (nome, cep, rua, numero, bairro, id_cidade)

        VALUES ('Terminal Secundário ' || random(), '60000-000', 'Rua B', '12', 'Bairro',
v_id_cidade);

END LOOP;

```

```

-----

-- 8. POPULAR TABELA: onibus (130 Ônibus)

-----

```

```

FOR i IN 1..130 LOOP

    v_placa := CHR(65 + (random()*25)::int) || CHR(65 + (random()*25)::int) ||
CHR(65 + (random()*25)::int) || '-' || CAST(CAST(random() * 8999 AS INT) + 1000
AS TEXT);

    SELECT id_viacao INTO v_id_viacao FROM viacao OFFSET (i % 120) LIMIT
1;

    INSERT INTO onibus (placa, capacidade, tipo, id_viacao)

    VALUES (

        v_placa,

        42,

        (ARRAY['Convencional', 'Executivo', 'Semi-Leito', 'Leito', 'Leito-Cama'])[1 +
floor(random() * 5)]::tipo_onibus,

        v_id_viacao

    );

END LOOP;

```

-- 9. POPULAR TABELA: assento (42 assentos por ônibus -> > 5000 entradas)

```
FOR v_id_onibus IN SELECT id_onibus FROM onibus LOOP
  FOR j IN 1..42 LOOP
    INSERT INTO assento (id_onibus, numero_assento)
      VALUES (v_id_onibus, LPAD(j::TEXT, 2, '0'));
  END LOOP;
END LOOP;
```

-- 10. POPULAR TABELA: viagem (125 Viagens)

```
FOR i IN 1..125 LOOP
  SELECT id_onibus INTO v_id_onibus FROM onibus OFFSET (i-1) LIMIT 1;
  INSERT INTO viagem (status, id_onibus)
    VALUES (
      (ARRAY['Agendada', 'Em Andamento', 'Concluida'])[1 + floor(random() *
3)]::status_viagem,
      v_id_onibus
    );
END LOOP;
```

-- 11. POPULAR TABELA: viagem_motorista (Vincula motoristas às viagens)

```
c_motoristas := ARRAY(SELECT cpf FROM motorista);
i := 1;
```

```

FOR v_id_viagem IN SELECT id_viagem FROM viagem LOOP

    SELECT m.cpf INTO v_cpf

    FROM motorista m

    JOIN viacao v ON m.id_viacao = v.id_viacao

    JOIN onibus o ON o.id_viacao = v.id_viacao

    JOIN viagem vi ON vi.id_onibus = o.id_onibus

    WHERE vi.id_viagem = v_id_viagem LIMIT 1;

    IF v_cpf IS NULL THEN

        v_cpf := c_motoristas[1 + (i % array_length(c_motoristas, 1))];

    END IF;

    INSERT INTO viagem_motorista (id_viagem, cpf_motorista) VALUES
(v_id_viagem, v_cpf) ON CONFLICT DO NOTHING;

    IF i <= 30 THEN

        v_cpf := c_motoristas[1 + ((i+5) % array_length(c_motoristas, 1))];

        INSERT INTO viagem_motorista (id_viagem, cpf_motorista) VALUES
(v_id_viagem, v_cpf) ON CONFLICT DO NOTHING;

    END IF;

    i := i + 1;

END LOOP;

-----

-- 12. POPULAR TABELA: trecho (Viagens longas com 12 a 15 trechos
sequenciais)

-----

c_rodoviaras := ARRAY(SELECT id_rodoviaria FROM rodoviaria);

i := 1;

```

```

FOR v_id_viagem IN SELECT id_viagem FROM viagem LOOP
    IF i <= 25 THEN
        v_num_trechos := CAST(random() * 3 AS INT) + 12;
    ELSE
        v_num_trechos := CAST(random() * 3 AS INT) + 2;
    END IF;

    v_data_atual := NOW() + (i || ' days')::interval;
    v_origem_rod := c_rodoviaras[1 + (i % array_length(c_rodoviaras, 1))];

    FOR t IN 1..v_num_trechos LOOP
        v_destino_rod := c_rodoviaras[1 + ((i + t) % array_length(c_rodoviaras,
1))];

        IF v_origem_rod = v_destino_rod THEN
            v_destino_rod := c_rodoviaras[1 + ((i + t + 1) %
array_length(c_rodoviaras, 1))];
        END IF;

        v_distancia := CAST(100 + (random() * 150) AS DECIMAL(6,2));
        v_preco := CAST((v_distancia * 0.35) AS DECIMAL(10,2));

        INSERT INTO trecho (id_viagem, ordem, distancia, data_hora_saida,
data_hora_chegada, preco_trecho, id_origem, id_destino)
            VALUES (
                v_id_viagem,
                t,
                v_distancia,
                v_data_atual,
                v_data_atual + (v_distancia / 60 || ' hours')::interval,

```

```

        v_preco,
        v_origem_rod,
        v_destino_rod
    );

    v_data_atual := v_data_atual + (v_distancia / 60 || ' hours')::interval + '30
minutes'::interval;

    v_origem_rod := v_destino_rod;

END LOOP;

i := i + 1;

END LOOP;

-----

-- 13. POPULAR TABELA: passagem (140 Passagens vendidas)

-----

c_usuarios := ARRAY(SELECT cpf FROM usuario);

i := 1;

FOR v_id_viagem IN SELECT id_viagem FROM viagem LOOP
    FOR j IN 1..2 LOOP
        v_cpf := c_usuarios[1 + ((i + j) % array_length(c_usuarios, 1))];

        INSERT INTO passagem (data_compra, valor_total, status, cpf_usuario,
id_viagem)
        VALUES (
            NOW() - '5 days'::interval,
            0.00,
            (ARRAY['Confirmada', 'Pendente', 'Utilizada'])[1 + floor(random() *
3)]::status_passagem,
            v_cpf,

```

```

        v_id_viagem

    ) RETURNING id_passagem INTO v_id_passagem;

-----

-- 14. RELACIONAMENTO TERNÁRIO: reserva
-----

    SELECT id_onibus INTO v_id_onibus FROM viagem WHERE id_viagem =
v_id_viagem;

    c_trechos_viagem := ARRAY(SELECT id_trecho FROM trecho WHERE
id_viagem = v_id_viagem ORDER BY ordem);

    IF array_length(c_trechos_viagem, 1) >= 2 THEN
        FOR k IN 1..2 LOOP
            v_id_trecho := c_trechos_viagem[k];

            INSERT INTO reserva (id_passagem, id_trecho, id_onibus,
numero_assento)
                VALUES (
                    v_id_passagem,
                    v_id_trecho,
                    v_id_onibus,
                    LPAD((10 + j)::TEXT, 2, '0')
                ) ON CONFLICT DO NOTHING;

            UPDATE passagem
                SET valor_total = valor_total + (SELECT preco_trecho FROM trecho
WHERE id_trecho = v_id_trecho)
                WHERE id_passagem = v_id_passagem;

        END LOOP;

```



```
END IF;  
  
i := i + 1;  
EXIT WHEN i > 140;  
END LOOP;  
EXIT WHEN i > 140;  
END LOOP;  
END $$;
```